LARAVEL FIELD GUIDE / 01

Eloquent ORM

Models turn database records into PHP objects. Build a query with filters and ordering, then call a method such as get() or firstOrFail() to retrieve the results.

These examples use a fictional library. Models, tables, fields, and views are assumed to exist. In the capstone workshop, that infrastructure is already prepared.

Expression	What it does
`Book::all()`	Returns every book as a collection.
`Book::findOrFail($id)`	Returns one book by primary key, or HTTP 404.
`Book::where('available', true)->get()`	Filters rows, then returns a collection.
`Book::orderBy('title')->get()`	Returns books sorted by title.
`Book::where('isbn', $isbn)->firstOrFail()`	Returns the first match, or HTTP 404.
`Book::create($validated)`	Inserts a record and returns the new model.
`$book->update($validated)`	Saves new values on an existing model.
`$book->delete()`	Deletes that model record.

Filter a calendar date, regardless of the time

A librarian wants loans whose return date is the next calendar day. A date-only comparison includes both morning and afternoon deadlines.

```
<?php
namespace App\Http\Controllers;

use App\Models\LibraryLoan;

class LoanController extends Controller
{
    public function index()
    {
        $loans = LibraryLoan::whereDate(
            'return_due_at', today()->addDay()
        )->orderBy('return_due_at')->get();

        return view('library.due-soon', compact('loans'));
    }
}
```

Read the chain: `whereDate()` narrows the rows; `orderBy()` sorts them; `get()` executes the query. `today()->addDay()` handles month and year changes. Use the application's timezone consistently.

Adapt the pattern: identify your model, date column, target date, and expected number of results. Filtering inside the database keeps unrelated rows out of the view.

LARAVEL FIELD GUIDE / 02

Validation & saving data

`$request->validate()` checks incoming data before it is saved. On success, it returns validated fields. For a normal web form, a failure redirects back with errors and old input.

Rule	Meaning
`required`	The field must be present and nonempty.
`string`	The value must be text.
`numeric / integer`	Accept a number / a whole number.
`min:10 / max:255`	For strings: character limits. For numbers: numeric limits.
`between:1,8`	Inclusive bounds; combine with numeric or integer for numbers.
`nullable`	Allow null when the field is optional.
`in:small,medium,large`	Accept only one of these exact values.

Example: a meal-order form

A kitchen accepts 1-8 meals per order. The customer provides the name and quantity, but the server decides the initial status.

```
<?php
namespace App\Http\Controllers;

use App\Models\MealOrder;
use Illuminate\Http\Request;

class MealOrderController extends Controller
{
    public function store(Request $request)
    {
        $validated = $request->validate([
            'customer_name' => ['required', 'string', 'min:2', 'max:80'],
            'quantity' => ['required', 'integer', 'between:1,8'],
            'notes' => ['nullable', 'string', 'max:500'],
        ]);

        $order = MealOrder::create([
            ...$validated,
            'status' => 'Queued',
        ]);

        return view('meals.receipt', compact('order'));
    }
}
```

Eloquent setup: the model's `$fillable` list must allow customer_name, quantity, notes, and status. In the workshop, model setup is already supplied.

Common mistakes: using `nullable` for a mandatory field; forgetting numeric typing before number bounds; saving raw input before validation; accepting a user-supplied status. Server rules still apply when browser validation is bypassed.

Check both paths: submit valid values, then try an empty name, quantity 0, quantity 9, and text in quantity. A rejected submission should leave saved data unchanged.

LARAVEL FIELD GUIDE / 03

Authentication & authorization

Authentication: who is signed in? Authorization: may this person perform this action? A Gate is a named server-side authorization rule.

Expression	Meaning
`auth()->check()`	Whether a user is authenticated.
`auth()->user()`	The authenticated User, or null for a guest.
`Gate::allows('view-restoration-notes')`	Checks permission and returns a boolean.
`Gate::authorize('view-restoration-notes')`	Continues if allowed; otherwise throws an authorization exception.

Example: protect a digital archive

Only an archivist may open private restoration notes. Define the rule when the application boots; Laravel supplies the authenticated user to the callback.

```
<?php
namespace App\Providers;

use App\Models\User;
use Illuminate\Support\Facades\Gate;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    public function boot(): void
    {
        Gate::define('view-restoration-notes', function (User $user): bool {
            return $user->role === 'archivist';
        });
    }
}
```

Enforce the rule before retrieving protected data

```
<?php
namespace App\Http\Controllers;

use App\Models\Document;
use Illuminate\Support\Facades\Gate;

class DocumentController extends Controller
{
    public function show(int $id)
    {
        Gate::authorize('view-restoration-notes');
        $document = Document::findOrFail($id);

        return view('documents.restoration', compact('document'));
    }
}
```

Check both identities: the archivist should receive the page; another role should receive HTTP 403, including through a direct URL. With this non-nullable User callback, guests are denied too. Hiding a link in Blade is only a UI choice; it does not protect the route.

LARAVEL FIELD GUIDE / 04

PHP conditions & match

`match` converts an input into a result. It compares values strictly, uses => for each arm, and returns the selected value. A `default` arm supplies a fallback.

Example: delivery choices become service labels

```
<?php
namespace App\Http\Controllers;

use App\Models\DeliveryEstimate;
use Illuminate\Http\Request;

class DeliveryEstimateController extends Controller
{
    public function store(Request $request)
    {
        $validated = $request->validate([
            'service' => ['required', 'in:standard,express,pickup'],
        ]);

        $label = match ($validated['service']) {
            'standard' => 'Three to five business days',
            'express' => 'Next business day',
            'pickup' => 'Collect in store',
            default => 'Contact the store',
        };

        $estimate = DeliveryEstimate::create([
            'service' => $validated['service'],
            'label' => $label,
        ]);

        return view('delivery.report', compact('estimate'));
    }
}
```

How to adapt: keep the input spelling exactly as submitted by the form. Change the input/output pairs for your own domain. The model must allow service and label in `$fillable`. Saving the result makes it available on later requests; displaying a temporary string alone does not.

Useful details to remember

Strict matching: a text input such as `express` matches the quoted string exactly. Different capitalization is a different value. An integer and a numeric string are different too.

One value out: assign the expression to a variable, then use that variable when creating a record or preparing a view. Use commas between arms and a semicolon after the closing brace.

Fallback vs validation: validation decides which inputs are accepted. The default arm handles any remaining value if the accepted input set changes later. In this example, validation currently admits only the three listed services.

Try it: choose every available service. Check the newest saved record each time, then refresh the report. A successful submission alone does not prove the mapping is correct.

LARAVEL FIELD GUIDE / 05

Run, inspect & test

Run commands from the project folder. Keep Docker Desktop running. Under WSL, use the Ubuntu terminal; on macOS, use Terminal. The launcher handles the PHP runtime and dependencies.

Commands from the project folder

Command	Purpose
`sh workshop.sh`	Start Docker setup (WSL, macOS, Linux).
`sh workshop.sh test BusTest`	Run one test class; replace BusTest as needed.
`sh workshop.sh seed`	Refresh sample dates for a new workshop day.
`sh workshop.sh stop`	Stop containers while preserving workshop data.
`docker compose exec app workshop-artisan route:list`	Inspect routes inside Docker.
`php artisan test --filter=BusTest`	Native PHP setup only: run one test class.

Available test classes: `AdvisorTest`, `BankingTest`, `BusTest`, `CityTest`, `NewsroomTest`. Keep Docker Desktop running. For Windows PowerShell outside WSL, use the invocation below (add `test BusTest` to run a test).

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\workshop.ps1
```

Workshop routine: read the TODO, adapt a relevant example, save, refresh, then test accepted and rejected cases. Stay on the student branch while solving the tasks.